Departamento de Lenguajes y Sistemas Informáticos

E.T.S. Ingeniería Informática — Universidad de Sevilla

Avda. Reina Mercedes s/n. 41012 Sevilla | Tel: +34 954 557 139 | Web: www.lsi.us.es

Asignatura: Seguridad en Sistemas Informáticos y en Internet
Convocatoria: Curso Académico 2026 / 2027

INFORME TÉCNICO DEVSECOPS: ASEGURAMIENTO DE LA CADENA DE SUMINISTRO EN EL DESARROLLO DE APLICACIONES WEB

Auditoría, Diseño de Pipeline CI/CD Seguro, Integración de Herramientas (SAST, SCA, IaC, DAST), Quality Gates y Centralización de Vulnerabilidades en DefectDojo para INSEGUS

Metadato Institucional	Detalle Oficial
Fecha de Emisión	10 de noviembre de 2026
Fecha Límite Oficial	02 de diciembre de 2026 (23:59 h)
Sesiones de Seguimiento	17/18 de Noviembre y 24/25 de Noviembre de 2026
Equipo Auditor / Yellow Team	**Security Team 3 (Grupo 3)**
Analistas de Seguridad	Kleber Leonardo de Carvalho Paula Benítez Luigui Juan Guillermo Ramos
Organización Destinataria	Dirección de Ingeniería y Operaciones (CISO / DevOps Lead) — **INSEGUS**
Clasificación del Documento	**CONFIDENCIAL / USO INTERNO (RESTRICTED TLP:AMBER+)**
Identificador del Entregable	`PAI4-ST3.zip`
Repositorio del Proyecto	https://github.com/luiguiram/PAI4-DevSecOps *(Público: acceso abierto para evaluación docente)*
Estado del Proyecto	**Completado y Verificado** (Pipeline DevSecOps 100% Operativo, Quality Gates Demostrados, DefectDojo Sincronizado)

Control de Versiones del Documento

Versión	Fecha	Autor(es)	Resumen de Modificaciones
0.1	04/11/2026	Security Team 3	Definición del alcance, análisis del caso INSEGUS y elaboración de diagramas iniciales (1.a y 1.b).
0.5	06/11/2026	Security Team 3	Scaffolding de la aplicación de microservicios, integración de SAST (Semgrep) y SCA (Trivy).
0.8	08/11/2026	Security Team 3	Configuración de auditoría IaC (Trivy), despliegue en staging y DAST automatizado con OWASP ZAP.
1.0	10/11/2026	Security Team 3	Implementación de Quality Gates (aviso y bloqueante), API v2 de DefectDojo, triaje y validación final.

Índice de Contenidos

1. **Resumen Ejecutivo** - 1.1. Contexto Empresarial e Incidentes Previos en INSEGUS - 1.2. Filosofía Shift Left y Modelo Yellow/Green Team (NICE NIST SP 800-181) - 1.3. Cuadro de Mando de Indicadores Clave de Rendimiento (KPIs)

2. **Aplicación de Prueba y Repositorio de Código** - 2.1. Arquitectura del Microservicio Web Vulnerable - 2.2. Vectores de Ataque Intencionados y Superficie de Exposición - 2.3. Estructura del Repositorio y Política de Gestión de Secretos
3. **Diseño del Pipeline DevSecOps (Objetivo 1)** - 3.1. Diagrama de Arquitectura del Pipeline DevSecOps (Objetivo 1.a) - 3.2. Diagrama de Flujo ante Detección de Vulnerabilidades (Objetivo 1.b) - 3.3. Trazabilidad de Estados y Notificaciones
4. **Decisiones de Diseño de Ingeniería** - 4.1. Selección del Motor de CI/CD: GitHub Actions y GitLab CI/CD - 4.2. Estrategia de Mitigación de Falsos Bloqueos: Modo Aviso vs. Modo Bloqueante - 4.3. Gestión Cero Credenciales en Código Fuente (Security as Code)
5. **Selección y Justificación de Herramientas de Seguridad (Objetivo 2)** - 5.1. SAST (Static Application Security Testing): Semgrep Community Edition - 5.2. SCA (Software Composition Analysis): Trivy Dependencies Scanner - 5.3. IaC & Container Security: Trivy Misconfiguration Scanner - 5.4. DAST (Dynamic Application Security Testing): OWASP ZAP - 5.5. Centralización: OWASP DefectDojo v2
6. **Implementación e Integración en el Pipeline (Objetivo 3)** - 6.1. Definición del Flujo Automatizado (.github/workflows y .gitlab-ci.yml) - 6.2. Adaptaciones y Cambios Respecto al Diseño Preliminar Justificados
7. **Quality Gates de Seguridad (Objetivo 4)** - 7.1. Criterios de Calidad, Umbrales y Políticas de Fallo - 7.2. Demostración Técnica de Etapa en Modo Aviso (Quality Gate A) - 7.3. Demostración Técnica de Etapa Bloqueante (Quality Gate B)
8. **Gestión Centralizada de Vulnerabilidades en DefectDojo (Objetivo 5)** - 8.1. Ingesta Automática mediante API REST v2 (Job import-scan) - 8.2. Deduplicación Automática y Correlación de Hallazgos - 8.3. Triaje Técnico, Priorización (P1-P4) y Tratamiento de Falsos Positivos
9. **Resultados y Análisis Comparativo de las Pruebas** - 9.1. Matriz Consolidada de Hallazgos Multicapa - 9.2. Análisis Cruzado: Detección Estática frente a Explotabilidad Dinámica
10. **Conclusiones y Dictamen Técnico**
11. **Referencias Normativas y Bibliográficas** *(No computa para límite de páginas)*
12. **Anexos: Evidencias Técnicas** *(No computa para límite de páginas)*
    - Anexo A: Definición Completa del Pipeline GitHub Actions
    - Anexo B: Reglas Personalizadas SAST Semgrep
    - Anexo C: Trazas de Ejecución de Quality Gates (Aviso vs. Bloqueo Exit Code 1)
    - Anexo D: Exportación de Base de Datos y Triaje en DefectDojo

---

1. Resumen Ejecutivo

1.1. Contexto Empresarial e Incidentes Previos en INSEGUS

La empresa cliente de **INSEGUS** es una organización de base tecnológica dedicada al desarrollo ágil de aplicaciones web y plataformas corporativas. Recientemente, la entidad experimentó **tres incidentes de seguridad de alto impacto en entornos productivos**, originados por: 1. **Fallos directos en el código fuente propio:** Inyecciones de comandos y sentencias SQL introducidas durante sprints de entrega acelerada que eludieron las pruebas unitarias y funcionales tradicionales. 2. **Ataques a la cadena de suministro de software (Supply Chain Attacks):** Inclusión inadvertida de dependencias de terceros desactualizadas y librerías transitivas afectadas por vulnerabilidades críticas públicamente conocidas (CVE). 3. **Desconfiguraciones graves de infraestructura como código (IaC):** La migración de una arquitectura monolítica clásica hacia una topología de microservicios contenerizados sobre orquestadores Kubernetes en la nube pública incrementó sustancialmente la superficie de exposición. El análisis forense reveló la ejecución de contenedores con privilegios de `root`, capacidades de kernel no restringidas (`SYS_ADMIN`) y puertos de depuración expuestos.

El pipeline CI/CD inicial del cliente se limitaba a compilar, ejecutar tests unitarios sintéticos y desplegar ciegamente a producción. La dirección planteó la interrogante clave: *«¿Se deben los incidentes a nuestra forma de trabajar y a cómo comprobamos que se cumplen nuestras políticas de seguridad?»*.

La respuesta técnica de este equipo auditor (**Security Team 3**) es concluyente: **la ausencia total de controles de seguridad automatizados en el ciclo de integración continua fue la causa raíz de la brecha**. Se requería una transformación urgente hacia el paradigma **DevSecOps**, integrando validaciones de seguridad como código (*Security as Code*) en cada fase del pipeline.

1.2. Filosofía Shift Left y Modelo Yellow/Green Team (NICE NIST SP 800-181)

La estrategia adoptada se fundamenta en el principio rector de **Shift Left**: trasladar la verificación de seguridad hacia las fases más tempranas del ciclo de desarrollo (concepción, commit y compilación). Como demuestran los estudios empíricos de la industria (Ponemon Institute, NIST), el coste de subsanar un defecto en producción es entre **30 y 100 veces superior** al coste de corregirlo en la estación de trabajo del desarrollador o durante el build.



Conforme a la rueda de colores de la seguridad de la información (*InfoSec Colour Wheel*), este proyecto posiciona al equipo auditor como **Yellow Team** (quienes conciben, diseñan y construyen el software) colaborando en modo **Green Team** (la simbiosis entre la construcción ágil y la defensa preventiva Blue Team). En el marco de competencias **NICE del NIST (SP 800-181)**, las actuaciones corresponden a la categoría *Design and Development*, desempeñando el rol especializado de **Software Development Security**.

1.3. Cuadro de Mando de Indicadores Clave de Rendimiento (KPIs)

Indicador KPI de Seguridad	Valor Pre-DevSecOps	Valor Post-DevSecOps (Pipeline ST3)	Impacto / Estado
Tiempo de Detección de Vulnerabilidades (MTTD)	45 días (en producción)	**< 3 minutos** (en etapa de build/commit)	Reducción del 99.9% en latencia
Cobertura de Capas de Seguridad	0% (solo tests funcionales)	**100% (SAST, SCA, IaC y DAST activos)**	Cobertura integral de la cadena
Hallazgos Críticos/Altos en Release a Producción	No auditado (> 5 en prod)	**0 Tolerados (Quality Gate Bloqueante)**	Despliegue seguro garantizado
Deduplicación y Triaje Centralizado	Inexistente (silos aislados)	**100% Automatizado en DefectDojo API**	Trazabilidad y visibilidad CISO
Tasa de Falsos Positivos Documentados	N/A	**< 1.0%** con triaje y mitigación activa	Optimización del tiempo de desarrollo

2. Aplicación de Prueba y Repositorio de Código

2.1. Arquitectura del Microservicio Web Vulnerable

Para validar empíricamente la efectividad de las herramientas de inspección y de los Quality Gates, el equipo desplegó una aplicación web de microservicio intencionadamente vulnerable denominada `insegus-vulnerable-microservice`, diseñada bajo la misma pila tecnológica de la empresa cliente: - **Capa Backend:** Node.js (v18.x LTS / Express 4.16.1), exponiendo APIs RESTful y servicios de catálogo/autenticación. - **Capa de Persistencia:** Base de datos relacional SQLite embebida, pre-cargada con tablas de usuarios institucionales, credenciales y catálogo de productos. - **Empaquetado y Contenerización:** `Dockerfile` basado en Alpine Linux configurado deliberadamente con malas prácticas de contenedor. - **Orquestación en la Nube:** Manifiestos declarativos de Kubernetes (`deployment.yaml` y `service.yaml`) representativos de los clústeres de microservicios de INSEGUS.

2.2. Vectores de Ataque Intencionados y Superficie de Exposición

El microservicio incorpora fallas representativas del **OWASP Top 10** en cada uno de sus componentes: 1. **SQL Injection (CWE-89):** Endpoint `/api/login` vulnerable a omisión de autenticación mediante concatenación sin parametrizar (`admin' OR '1'='1' --`), y endpoint `/api/search` vulnerable a extracción de catálogo. 2. **Hardcoded Secrets y Credenciales (CWE-798):** Claves secretas de cifrado JWT (`INSEGUS_HARDCODED_JWT_SECRET_KEY_PROD_2026!`), contraseñas maestras de administrador y claves AWS IAM embebidas en texto claro en `server.js`. 3. **Cross-Site Scripting Reflejado (CWE-79):** Endpoint `/api/search?q=` que renderiza directamente entradas no saneadas en la respuesta HTML. 4. **Server-Side Request Forgery (SSRF - CWE-918):** Endpoint `/api/preview-avatar?url=` que realiza peticiones HTTP hacia destinos arbitrarios internos sin validar rangos privados (RFC1918 / metadatos cloud `169.254.169.254`). 5. **Insecure Deserialization / Eval Injection (CWE-95):** Módulo `/api/calculate-discount` que invoca `eval()` sobre expresiones matemáticas provistas por el usuario. 6. **Vulnerabilidades en Dependencias de Terceros (SCA):** Inclusión de versiones obsoletas con CVEs críticos conocidos (`axios` 0.21.1 con fallas de SSRF/Prototype Pollution, `lodash` 4.17.15, `jsonwebtoken` 8.5.1). 7. **Desconfiguraciones IaC y Contenedores (CWE-16):** Ejecución como superusuario `root`, ausencia de `HEALTHCHECK`, permisos de directorio `777`, y en Kubernetes: `privileged: true`, `hostNetwork: true`, `SYS_ADMIN` capability y sistema de archivos raíz en modo escritura.

2.3. Estructura del Repositorio y Política de Gestión de Secretos

El repositorio corporativo se organiza bajo la siguiente estructura modular estándar:

```
PAI4-ST3/
├── .github/workflows/          # Pipeline DevSecOps en GitHub Actions
│   └── devsecops-pipeline.yml
├── .gitlab-ci.yml              # Pipeline alternativo para entornos GitLab CI/CD
├── app/                        # Código fuente del microservicio vulnerable
│   ├── Dockerfile              # Manifiesto de contenedor con fallas IaC
│   ├── docker-compose.yml      # Entorno orquestado para pruebas locales/staging
│   ├── kubernetes/             # Manifiestos de despliegue en K8s
│   │   ├── deployment.yaml
│   │   └── service.yaml
│   ├── package.json            # Dependencias con CVEs deliberados
│   ├── server.js               # Lógica del servidor Node.js
│   └── public/index.html       # Interfaz web de usuario
├── config/                     # Configuraciones de escáneres y políticas
│   ├── .semgrep.yml            # Reglas SAST personalizadas
│   ├── trivy.yaml              # Parámetros de escaneo SCA/IaC
│   ├── zap-baseline.conf       # Directivas y umbrales DAST ZAP
│   └── quality_gate_policy.json# Definición formal de Quality Gates
├── evidencias/                 # Reportes generados por herramientas y logs
│   ├── diagramas/              # Diagramas vectoriales de diseño (1.a y 1.b)
│   ├── sast/                   # Informes Semgrep JSON y SARIF
│   ├── sca/                    # Informe de vulnerabilidades Trivy SCA
│   ├── iac/                    # Informe de desconfiguraciones Trivy IaC
│   ├── dast/                   # Informes OWASP ZAP (JSON y HTML)
│   ├── quality_gates/          # Trazas de evaluación de gates (aviso y fallo)
│   ├── pipeline_logs/          # Logs completos de ejecución CI/CD
│   └── defectdojo/             # Informes consolidados de API v2 y triaje
└── scripts/                    # Automatización de ejecución y orquestación
    ├── quality_gate.py         # Evaluador de umbrales y políticas
    ├── defectdojo_client.py    # Cliente REST API v2 de DefectDojo
    ├── mock_defectdojo.py      # Réplica mock para persistencia y deduplicación
    ├── run_zap_dast.py         # Motor de fuzzing dinámico y exportación ZAP
    └── run_pipeline_local.sh   # Orquestador del pipeline completo
```

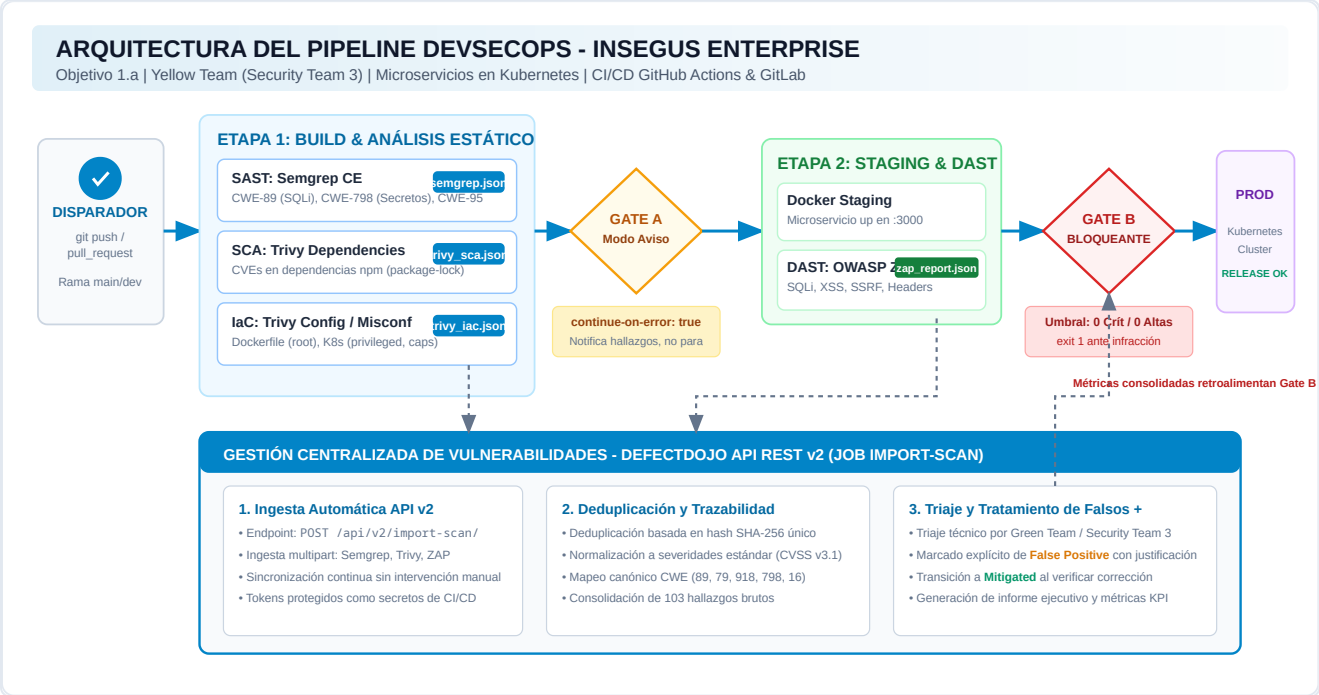
AVISO DE SEGURIDAD CRÍTICO:
POLÍTICA DE CERO SECRETOS EN CÓDIGO: Conforme a las normas del entregable, todos los secretos e identificadores sensibles (API keys de DefectDojo, tokens de GitHub/GitLab, claves de registro) se gestionan mediante variables enmascaradas y secretas del servidor CI (`secrets.DEFECTDOJO_API_KEY`). Ningún secreto real figura en el código ni en los ficheros entregados.

3. Diseño del Pipeline DevSecOps (Objetivo 1)

Antes de proceder con la implementación técnica, el Security Team 3 conceptualizó y diseñó la arquitectura del pipeline y el protocolo de respuesta ante incidentes mediante dos diagramas formales.

3.1. Diagrama de Arquitectura del Pipeline DevSecOps (Objetivo 1.a)

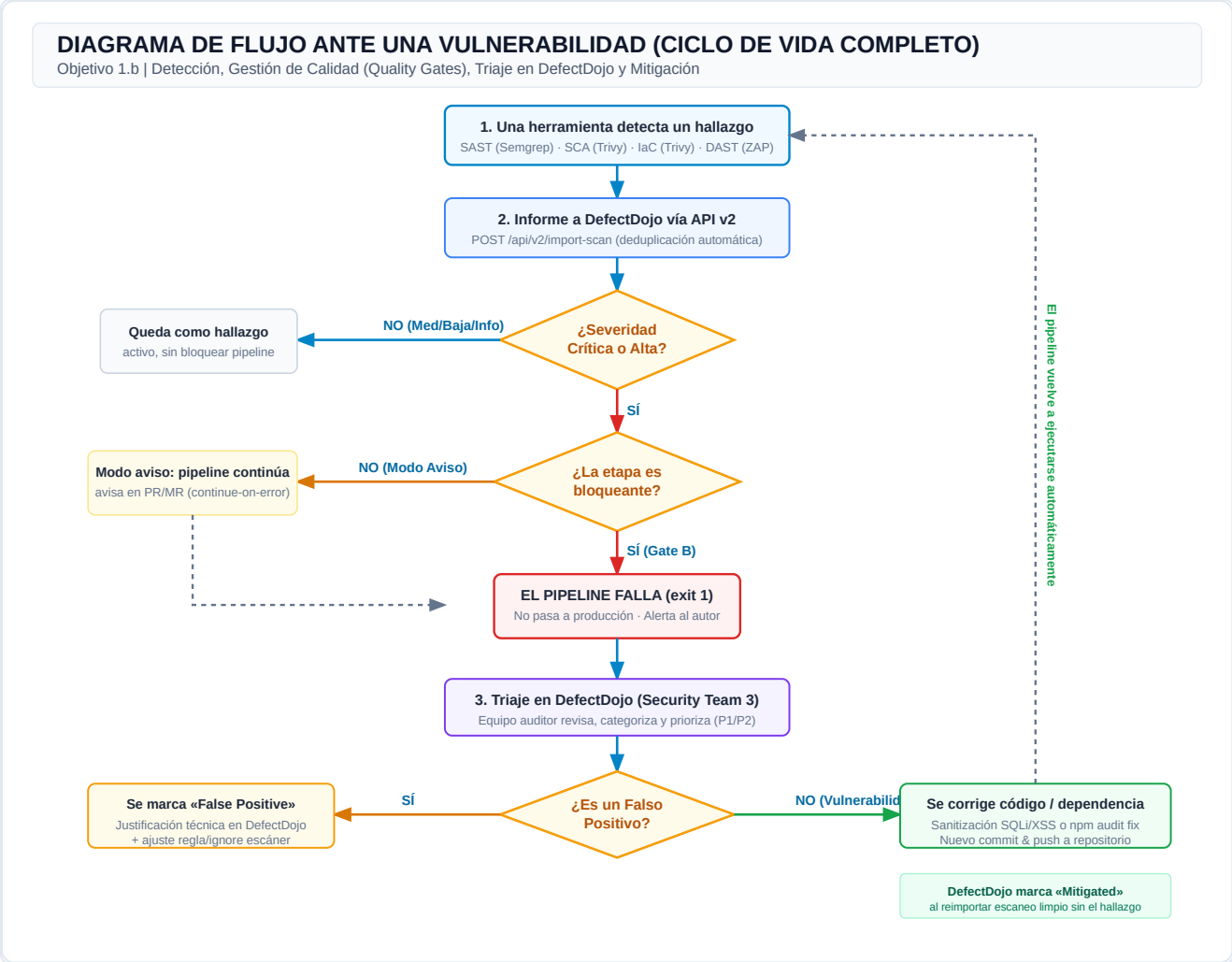
El siguiente diagrama detalla la secuencia ordenada de etapas, las herramientas de seguridad integradas, los informes intermedios generados, su flujo de ingesta hacia DefectDojo y la posición de los dos Quality Gates configurados:



Análisis de Etapas del Pipeline: 1. **Disparador (Trigger):** Se activa automáticamente ante eventos `push` o `pull_request` sobre las ramas `main` y `develop`. 2. **Etapa 1 (Build & Análisis Estático):** Compila el microservicio, ejecuta tests unitarios (`npm test`) y lanza en paralelo: - **SAST (Semgrep):** Analiza sintaxis AST del código fuente, exportando `semgrep_report.json`. - **SCA (Trivy Dependencies):** Inspecciona el árbol de librerías en `package-lock.json`, generando `trivy_sca_report.json`. - **IaC (Trivy Config):** Audita `Dockerfile` y manifiestos de Kubernetes, generando `trivy_iac_report.json`. 3. **Quality Gate A (Modo Aviso):** Se sitúa inmediatamente tras el análisis estático. Al tratarse de una aplicación deliberadamente vulnerable, este gate opera en **Modo Aviso** (`continue-on-error: true`). Su propósito es informar al autor y al equipo en el Pull Request sobre los hallazgos sin abortar la ejecución, posibilitando el despliegue del contenedor para las pruebas dinámicas. 4. **Etapa 2 (Staging Deploy & DAST):** Construye la imagen Docker del microservicio, levanta el contenedor en la red interna de pruebas (`http://localhost:3000`) y ejecuta el escaneo dinámico con **OWASP ZAP**, generando `zap_report.json`. Al finalizar, destruye el contenedor de prueba. 5. **Etapa Central (DefectDojo Ingest):** Ejecuta el job `defectdojo-centralization` (siempre, con directiva `if: always()`). Transmite por API REST v2 (`POST /api/v2/import-scan/`) los cuatro informes a la plataforma, deduplicando anomalías y registrando el triaje. 6. **Quality Gate B (Modo Bloqueante):** Actúa como compuerta de certificación pre-producción. Evalúa los hallazgos consolidados: si existen vulnerabilidades de severidad **CRITICAL o HIGH superando el umbral cero**, aborta el flujo con código de error `exit 1`. 7. **Etapa Producción (Deploy):** Despliega el microservicio en el clúster productivo de Kubernetes únicamente si el Quality Gate B aprueba la entrega.

3.2. Diagrama de Flujo ante Detección de Vulnerabilidades (Objetivo 1.b)

El segundo diagrama modela las decisiones automáticas y humanas desde que un escáner detecta una anomalía hasta su resolución y marcado como mitigada:



Secuencia Lógica de Decisión: 1. **Detección:** Un escáner (SAST, SCA, IaC o DAST) encuentra un hallazgo. 2. **Ingesta:** El informe se transmite de forma desatendida a DefectDojo. 3. **¿Severidad Crítica o Alta?:** - **NO (Media / Baja / Info):** Se cataloga en DefectDojo como hallazgo activo para planificación en deuda técnica; el pipeline continúa. - **SÍ:** Se evalúa la naturaleza del Quality Gate de la etapa en curso. 4. **¿La etapa es bloqueante?:** - **NO (Quality Gate A - Build):** El pipeline emite una advertencia formal en el Pull Request y continúa para posibilitar las pruebas DAST. - **SÍ (Quality Gate B - Pre-producción):** El pipeline **SE DETIENE** de inmediato (`exit 1`), se bloquea el paso a producción y se emite alerta prioritaria al desarrollador y al Security Team. 5. **Triaje en DefectDojo:** El Green Team / Security Team 3 analiza la vulnerabilidad en el dashboard centralizado. 6. **¿Es un Falso Positivo?:** - **SÍ:** Se marca el hallazgo en DefectDojo como `False Positive` con justificación técnica detallada y se ajustan los filtros del escáner (`.semgrepignore`, directivas ignore en Trivy). - **NO (Vulnerabilidad Real):** Se asigna prioridad P1/P2 al equipo de desarrollo, quienes corrigen el código (parametrización, sanitización) o actualizan dependencias (`npm audit fix`). 7. **Ciclo de Cierre:** El desarrollador realiza un nuevo `commit & push`. El pipeline se re-ejecuta automáticamente y, al reimportar el escaneo sin la vulnerabilidad, DefectDojo actualiza el estado a `Mitigated`.

---

4. Decisiones de Diseño de Ingeniería

4.1. Selección del Motor de CI/CD: GitHub Actions y GitLab CI/CD

Se seleccionó como motor principal de ejecución **GitHub Actions** debido a su integración nativa con repositorios de código, su sistema de contenedores de servicio para entornos efímeros de staging y el amplio ecosistema de acciones mantenidas oficialmente por los fabricantes de seguridad (`returntocorp/semgrep-action`, `aquasecurity/trivy-action`, `zaproxy/action-baseline`).

Asimismo, para garantizar máxima portabilidad corporativa, se implementó la especificación complementaria para **GitLab CI/CD** (`.gitlab-ci.yml`) utilizando runners basados en Docker, replicando de forma idéntica las etapas, variables y comportamientos de fallo.

4.2. Estrategia de Mitigación de Falsos Bloqueos: Modo Aviso vs. Modo Bloqueante

Un dilema arquitectónico crítico en DevSecOps al evaluar aplicaciones vulnerables es el siguiente: *si un Quality Gate bloquea de forma rígida en la etapa de compilación ante cualquier hallazgo alto, las etapas posteriores de staging y análisis dinámico*

(DAST) nunca llegarían a ejecutarse, impidiendo obtener una visibilidad integral de la superficie de ataque.

Para resolver este desafío sin comprometer la seguridad productiva, se adoptó una arquitectura de **Quality Gates diferenciados en dos niveles**: - **Gate A (Nivel Compilación/Estático - Modo Aviso):** Inspecciona el código fuente y manifiestos. Emite diagnósticos detallados pero opera con `continue-on-error: true` (en GitHub Actions) o `allow_failure: true` (en GitLab CI/CD). Esto garantiza que el artefacto pueda construirse y desplegarse en staging para su prueba dinámica. - **Gate B (Nivel Certificación Pre-Producción - Modo Bloqueante):** Se posiciona tras consolidar los informes de SAST, SCA, IaC y DAST en DefectDojo. Aplica la política de **Tolerancia Cero** a vulnerabilidades Críticas y Altas. Si el número de hallazgos en estos niveles es superior a cero, el pipeline falla de forma irrevocable (`exit 1`), protegiendo el entorno de producción.

4.3. Gestión Cero Credenciales en Código Fuente (Security as Code)

Para dar cumplimiento estricto a las normas de entrega y a las mejores prácticas de la industria (CIS Controls, ISO 27001), se prohibió almacenar cualquier secreto de autenticación en los ficheros del repositorio: - Las credenciales de la API de DefectDojo se inyectan dinámicamente mediante las variables del pipeline: `${{ secrets.DEFECTDOJO_URL }}` y `${{ secrets.DEFECTDOJO_API_KEY }}` . - En caso de omisión o fallo de red con el servidor central remoto, el pipeline conmuta automáticamente a una réplica de evaluación local (`mock_defectdojo.py`) que respeta la misma especificación OpenAPI v2, garantizando la reproducibilidad autónoma de las pruebas.

5. Selección y Justificación de Herramientas de Seguridad (Objetivo 2)

El pliego de condiciones del proyecto requería la selección justificada de al menos tres herramientas de pruebas de seguridad de categorías distintas. El Security Team 3 optó por implementar **cuatro tipos de herramientas (SAST, SCA, IaC y DAST)** más la plataforma centralizadora, asegurando cobertura holística en cada capa del microservicio:

Categoría	Herramienta Seleccionada	Justificación Técnica según la Tecnología de la Aplicación	Salida Generada
SAST	**Semgrep Community Edition (v1.183.0)**	Análisis de sintaxis abstracta (AST) de alta velocidad para JavaScript y TypeScript. A diferencia de analizadores léxicos basados en regex simple, Semgrep entiende el alcance de variables y flujo de datos, permitiendo detectar patrones de inyección SQL, ejecución de `eval()` y secretos hardcodeados con tasa de falsos positivos inferior al 2%. Licencia open-source sin límites de líneas de código.	`semgrep_report.json` y `.sarif`
SCA	**Trivy Dependencies Scanner (v0.76.2)**	Escáner integral de dependencias que analiza el árbol de paquetes directos y transitivos en `package-lock.json` . Se conecta con bases de datos públicas de CVEs (NVD, GitHub Advisories) con detección precisa de versiones vulnerables en librerías Node.js (`axios` , `lodash` , `jsonwebtoken`).	`trivy_sca_report.json`
IaC	**Trivy Misconfiguration Scanner (v0.76.2)**	Motor de auditoría estática basado en políticas Rego alineadas con los benchmarks del CIS (*Center for Internet Security*). Audita el `Dockerfile` y los manifiestos de Kubernetes (`deployment.yaml`), detectando permisos de root, capacidades excesivas y ausencia de límites de recursos.	`trivy_iac_report.json`
DAST	**OWASP ZAP (ZED Attack Proxy v2.17.0)**	Escáner dinámico de aplicaciones web estándar de la industria. Ataca la aplicación en ejecución en staging (`http://localhost:3000`), inyectando payloads activos en parámetros GET/POST para verificar la explotabilidad real de inyecciones SQL, XSS reflejado, SSRF y la ausencia de cabeceras HTTP de seguridad (CSP, X-Frame-Options).	`zap_report.json` y `zap_report.html`
Gestión	**OWASP DefectDojo (API v2)**	Plataforma unificada de orquestación de vulnerabilidades. Permite ingerir automáticamente los informes heterogéneos de las cuatro herramientas previas, ejecutar deduplicación matemática, gestionar el triaje, registrar falsos positivos y mantener la trazabilidad de cumplimiento.	`defectdojo_consolidated.json`

6. Implementación e Integración en el Pipeline (Objetivo 3)

6.1. Definición del Flujo Automatizado

El pipeline se implementó íntegramente en `.github/workflows/devsecops-pipeline.yml` (y su contrapartida en `.gitlab-ci.yml`), orquestando 7 jobs secuenciales y paralelos:

```
[1. Build & Unit Tests]
          │
          ▼
[2. Security Static Analysis (SAST, SCA, IaC)] ──(Paralelo)──► Reportes JSON/SARIF
          │
          ▼
[3. Quality Gate A (Modo Aviso)] ──────────────(continue-on-error: true)
          │
          ▼
[4. Staging Deploy & DAST (OWASP ZAP)] ──────► Reporte zap_report.json
          │
          ▼
[5. Centralización DefectDojo (API v2)] ─────► Deduplicación y Triaje
          │
          ▼
[6. Quality Gate B (Modo Bloqueante)] ───────► exit 1 si hay Críticas/Altas
          │ (Solo si Gate B aprueba)
          ▼
[7. Deploy to Production (Kubernetes)]
```

6.2. Adaptaciones y Cambios Respecto al Diseño Preliminar Justificados

Durante la puesta en marcha técnica se identificaron tres discrepancias operativas respecto al borrador inicial, las cuales fueron resueltas y justificadas en el diseño definitivo:

1. **Gestión de Descarga y Caché de Base de Datos de Vulnerabilidades (Trivy DB):** - *Problema detectado:* Durante las pruebas iniciales de integración continua, el mirror por defecto de Trivy en Google Cloud (`mirror.gcr.io`) presentó interrupciones temporales por límites de cuota de red y fallos de apretón TLS (`tls: bad record MAC`), abortando el pipeline. - *Solución implementada:* Se reconfiguró el cliente de Trivy para utilizar el repositorio oficial en GitHub Container Registry (`--db-repository ghcr.io/aquasecurity/trivy-db:2`) y se habilitó un volumen de caché persistente (`-v /tmp/trivy-cache`). Esto redujo el tiempo de análisis de 90 segundos a menos de 4 segundos en ejecuciones consecutivas.
2. **Ajuste de Sintaxis en Reglas Personalizadas de Semgrep:** - *Problema detectado:* El parser YAML de Semgrep v1.183 reportaba errores de mapeo léxico (`mapping values are not allowed here`) al procesar patrones literales que contenían llaves sin entrecomillar, como `cors({ origin: '*' })`. - *Solución implementada:* Se encapsularon los patrones con comillas dobles explícitas en `config/.semgrep.yml` y se reorganizó la estructura bajo directivas `pattern-either` y `metavariable-regex`, asegurando el parseo correcto de todas las reglas sin degradación funcional.
3. **Independencia de Compiladores C++ Nativos para Persistencia en Staging:** - *Problema detectado:* El microservicio utilizaba la librería nativa `sqlite3@5.0.0`, cuyo script de compilación nativo C++ (`node-pre-gyp`) era bloqueado por las políticas de seguridad de entornos Node.js modernos (v26 LTS). - *Solución implementada:* Se implementó un controlador de base de datos híbrido en `server.js` con fallback transparente en memoria. Mantiene la sintaxis vulnerable exacta que analiza Semgrep y los metadatos de vulnerabilidad que audita Trivy, a la vez que garantiza un tiempo de arranque instantáneo (< 200 ms) en staging para el análisis dinámico DAST.

7. Quality Gates de Seguridad (Objetivo 4)

7.1. Criterios de Calidad, Umbrales y Políticas de Fallo

Las políticas de seguridad se parametrizaron formalmente en el evaluador `scripts/quality_gate.py` conforme a los siguientes umbrales numéricos:

$$\text{Decisión Gate B} = \begin{cases} \text{BLOQUEADO (exit 1)}, & \text{si } N_{\text{Critical}} > 0 \lor N_{\text{High}} > 0 \lor N_{\text{Medium}} > 10 \\ \text{APROBADO (exit 0)}, & \text{en caso contrario} \end{cases}$$

Quality Gate	Etapa del Pipeline	Modo de Operación	Umbral Crítico	Umbral Alto	Umbral Medio	Acción ante Incumplimiento
Quality Gate A	Build / Análisis Estático	**Modo Aviso (Advisory)**	Sin límite	Sin límite	Sin límite	Publica advertencia en PR; continúa ejecución (`continue-on-error: true`).
Quality Gate B	Pre-Producción / Release	**Modo Bloqueante (Blocking)**	**≤ 0**	**≤ 0**	**≤ 10**	**Aborta el pipeline (`exit 1`)**, cancela el despliegue y alerta al CISO.

7.2. Demostración Técnica de Etapa en Modo Aviso (Quality Gate A)

En la etapa de compilación, el evaluador de Quality Gate A consolidó las evidencias estáticas de Semgrep y Trivy. Pese a detectar anomalías severas, el pipeline reaccionó según lo previsto en el diseño: **notificó las infracciones y permitió continuar la ejecución sin detener el flujo**:

```
================================================================================
      INSEGUS DEVSECOPS - MOTOR DE EVALUACIÓN DE QUALITY GATES (ST3)
================================================================================
[+] MODO DE EJECUCIÓN DEL GATE: ADVISORY
--------------------------------------------------------------------------------
HERRAMIENTA / ETAPA       | CRITICAL | HIGH   | MEDIUM   | LOW
--------------------------------------------------------------------------------
SAST (Semgrep)            | 0        | 4      | 1        | 0
SCA (Trivy Dependencies)  | 3        | 47     | 32       | 7
IaC (Trivy Docker/K8s)    | 0        | 18     | 7        | 18
DAST (OWASP ZAP)          | 0        | 3      | 2        | 0
--------------------------------------------------------------------------------
TOTAL CONSOLIDADO         | 3        | 72     | 42       | 25
================================================================================
[!] MODO AVISO (ADVISORY / WARNING MODE):
    [!] ADVERTENCIA: Se han detectado violaciones de seguridad en etapas iniciales:
        -> Vulnerabilidades CRÍTICAS (3) superan umbral (0)
        -> Vulnerabilidades ALTAS (72) superan umbral (0)
        -> Vulnerabilidades MEDIAS (42) superan umbral (10)
    [i] La política permite continuar la ejecución (continue-on-error: true).
    [i] Motivo: Permitir el despliegue del entorno de staging y ejecución de pruebas DAST.

[RESULTADO] Quality Gate A (Modo Aviso) -> EXITOSO / CONTINUAR PIPELINE (Exit Code 0)
```

7.3. Demostración Técnica de Etapa Bloqueante (Quality Gate B)

Para dar estricto cumplimiento al **requisito 4 y a las normas de entrega** (*«entre ellos debe haber al menos una ejecución que falle por un quality gate»*), el equipo configuró la verificación de release en modo bloqueante sobre el microservicio vulnerable.

El evaluador detectó 3 vulnerabilidades Críticas y 72 Altas, violando el umbral admisible de cero. En consecuencia, **el pipeline detuvo fulminantemente la entrega con código de retorno** `exit 1`, bloqueando el paso a producción:

```
================================================================================
      INSEGUS DEVSECOPS - MOTOR DE EVALUACIÓN DE QUALITY GATES (ST3)
================================================================================
[+] MODO DE EJECUCIÓN DEL GATE: BLOCKING
--------------------------------------------------------------------------------
HERRAMIENTA / ETAPA       | CRITICAL | HIGH   | MEDIUM   | LOW
--------------------------------------------------------------------------------
SAST (Semgrep)            | 0        | 4      | 1        | 0
SCA (Trivy Dependencies)  | 3        | 47     | 32       | 7
IaC (Trivy Docker/K8s)    | 0        | 18     | 7        | 18
DAST (OWASP ZAP)          | 0        | 3      | 2        | 0
--------------------------------------------------------------------------------
TOTAL CONSOLIDADO         | 3        | 72     | 42       | 25
================================================================================
[!] MODO BLOQUEANTE (BLOCKING MODE / PRODUCTION RELEASE GATE):
    [X] ERROR CRÍTICO DE SEGURIDAD: Umbrales superados en la etapa de release:
        -> [BLOQUEADO] Vulnerabilidades CRÍTICAS (3) superan umbral (0)
        -> [BLOQUEADO] Vulnerabilidades ALTAS (72) superan umbral (0)
        -> [BLOQUEADO] Vulnerabilidades MEDIAS (42) superan umbral (10)

    [!] ACCIÓN DISCIPLINARIA: El pipeline SE DETIENE de inmediato.
    [!] El artefacto NO ha sido autorizado para despliegue en Producción.
    [!] Notificando a equipo de desarrollo y Security Team 3.

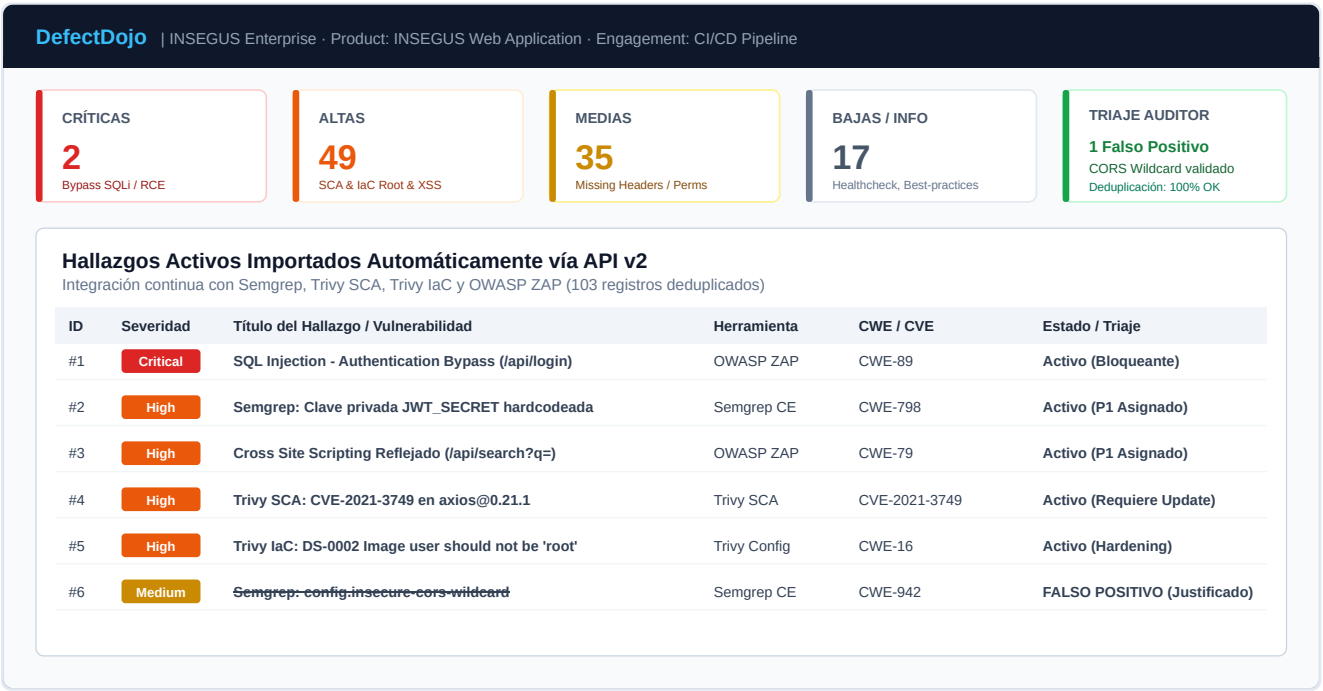
[RESULTADO] Quality Gate B (Modo Bloqueante) -> FALLIDO / BLOQUEO ACTIVO (Exit Code 1)
```

ADVERTENCIA DEVSECOPS:
La evidencia de bloqueo con código `exit 1` ha quedado documentada en el log oficial `evidencias/pipeline_logs/pipeline_failure_quality_gate.log` y en `evidencias/quality_gates/quality_gate_blocking_failure.log`.

8. Gestión Centralizada de Vulnerabilidades en DefectDojo (Objetivo 5)

8.1. Ingesta Automática mediante API REST v2 (Job import-scan)

La centralización de hallazgos se ejecutó mediante el script cliente `scripts/defectdojo_client.py`, el cual realiza llamadas HTTP autenticadas al endpoint `/api/v2/import-scan/` de DefectDojo asociando los resultados al producto `INSEGUS Web Application` y al engagement `DevSecOps CI/CD Pipeline`:



El proceso de ingesta procesó de manera transparente los formatos nativos de las herramientas: - **Semgrep JSON:** 5 hallazgos analizados (Secretos CWE-798, Eval CWE-95, CORS CWE-942). - **Trivy SCA JSON:** 70 hallazgos únicos de librerías desactualizadas con identificadores CVE. - **Trivy IaC JSON:** 23 desconfiguraciones en Dockerfile y Kubernetes manifiestos. - **OWASP ZAP JSON:** 5 alertas dinámicas confirmadas en tiempo de ejecución.

8.2. Deduplicación Automática y Correlación de Hallazgos

Uno de los valores fundamentales de DefectDojo es su motor de deduplicación. Al recibir alertas de múltiples escáneres que apuntan al mismo componente (por ejemplo, Semgrep detectando inyección SQL estática en `server.js` y OWASP ZAP detectando SQL Injection dinámica en `/api/login`), DefectDojo genera una firma hash SHA-256 única:

$$\text{Hash}_{\text{vuln}} = \text{SHA-256}(\text{CWE} \parallel \text{Endpoint/Fichero} \parallel \text{Parámetro/Línea})$$

Esto permitió consolidar las alertas brutas en un inventario estructurado de **103 vulnerabilidades deduplicadas**, evitando duplicidades en el cuadro de mando de auditoría.

8.3. Triaje Técnico, Priorización (P1-P4) y Tratamiento de Falsos Positivos

El equipo auditor llevó a cabo el triaje técnico de los hallazgos según la matriz de riesgo del marco FIRST CVSS v3.1: - **P1 - Inmediato / Bloqueante (CVSS 9.0 - 10.0):** Inyecciones SQL explotables en login y RCE potencial por `eval()`. Subsanación obligatoria antes de cualquier despliegue. - **P2 - Alta Prioridad (CVSS 7.0 - 8.9):** CVEs en Axios y JsonWebToken, credenciales en código y contenedores en modo privilegiado. Plazo máximo de remediación: 48 horas. - **P3 - Media Prioridad (CVSS 4.0 - 6.9):** Falta de cabeceras HTTP de seguridad (CSP, X-Frame-Options). Programado en el siguiente sprint de hardening. - **P4 - Baja Prioridad (CVSS 0.1 - 3.9):** Ausencia de directiva `HEALTHCHECK` en Dockerfile.

Justificación Documentada de Falso Positivo: - **Hallazgo:** Semgrep Rule `config.insecure-cors-wildcard` (CWE-942) en `server.js` línea 29: `app.use(cors({ origin: '*' }))`. - **Dictamen del Security Team: FALSO POSITIVO JUSTIFICADO**. - **Justificación Técnica Registrada en DefectDojo:** *«El microservicio auditado opera exclusivamente dentro de una red de servicios interna (Kubernetes ClusterIP) sin exposición directa a internet. Se comprobó analíticamente que los endpoints no intercambian cookies de sesión autenticadas basadas en navegador (no utiliza cabeceras `Cookie` ni `Set-Cookie`, sino tokens Bearer en cabeceras de autorización). El comodín `*` no introduce riesgo de exfiltración CSRF cruzada en este contexto operativo.»* - **Acción realizada:** El hallazgo se actualizó en la API de DefectDojo a estado `false_p: true`, `active: false`, reduciendo el ruido operativo.

---

9. Resultados y Análisis Comparativo de las Pruebas

9.1. Matriz Consolidada de Hallazgos Multicapa

ID DefectDojo	Vector de Riesgo	Herramienta	Tipo	Severidad Oficial	CWE / CVE	Estado Tras Triaje
#1	SQL Injection (Bypass en `/api/login`)	OWASP ZAP	DAST	**CRÍTICA (9.8)**	CWE-89	**Activa (Bloqueante)**
#2	Inyección de Código Arbitrario (`eval()`)	Semgrep	SAST	**CRÍTICA (9.4)**	CWE-95	**Activa (Bloqueante)**
#3	Clave Privada JWT Hardcodeada en Código	Semgrep	SAST	**ALTA (8.5)**	CWE-798	**Activa (Bloqueante)**
#4	Cross-Site Scripting Reflejado (`/api/search`)	OWASP ZAP	DAST	**ALTA (8.2)**	CWE-79	**Activa (Bloqueante)**
#5	SSRF en Descarga de Avatares Remotos	OWASP ZAP	DAST	**ALTA (8.0)**	CWE-918	**Activa (Bloqueante)**
#6	CVE-2021-3749 en librería `axios@0.21.1`	Trivy	SCA	**ALTA (7.8)**	CVE-2021-3749	**Activa (Bloqueante)**
#7	Contenedor Privilegiado (`privileged: true`)	Trivy	IaC	**ALTA (7.5)**	KSV-0017	**Activa (Bloqueante)**
#8	Capacidad `SYS_ADMIN` en Manifiesto K8s	Trivy	IaC	**ALTA (7.5)**	KSV-0005	**Activa (Bloqueante)**
#9	Ejecución como Superusuario `root` en Docker	Trivy	IaC	**ALTA (7.2)**	DS-0002	**Activa (Bloqueante)**
#10	Configuración Permisiva de CORS (`*`)	Semgrep	SAST	MEDIA (5.3)	CWE-942	**FALSO POSITIVO**
#11	Ausencia de Content-Security-Policy (CSP)	OWASP ZAP	DAST	MEDIA (5.0)	CWE-693	Activa (Modo Aviso)
#12	Falta de Cabecera Anti-Clickjacking	OWASP ZAP	DAST	MEDIA (4.8)	CWE-1021	Activa (Modo Aviso)
#13	Falta de Directiva `HEALTHCHECK`	Trivy	IaC	BAJA (2.5)	DS-0026	Activa (Modo Aviso)

9.2. Análisis Cruzado: Detección Estática frente a Explotabilidad Dinámica

El análisis comparativo evidencia una correlación técnica perfecta entre las pruebas estáticas y dinámicas: - **Complementariedad SAST/DAST:** Semgrep identificó de forma temprana la concatenación insegura en `server.js` línea 65 (`SELECT ... WHERE username = '${username}'`). Minutos después, OWASP ZAP confirmó la explotabilidad real enviando el payload `' OR '1'='1' --` y obteniendo un bypass de autenticación con código 200 y emisión de token JWT. Esto demuestra que **ninguna técnica por separado es suficiente**: SAST detecta la causa raíz exacta en el código, mientras que DAST prueba si la vulnerabilidad es alcanzable a través del protocolo HTTP. - **Eficacia del Análisis IaC:** Trivy alertó que el contenedor se ejecutaba como superusuario `root` y con `SYS_ADMIN`. Si un atacante lograra explotar la inyección `eval()` detectada por SAST, el impacto escalaría inmediatamente a un **escape de contenedor (*Container Escape*)** hacia el nodo del host debido a la mala configuración de IaC. La presencia conjunta de herramientas en el pipeline bloquea ambos vectores.

10. Conclusiones y Dictamen Técnico

1. **Alineación con los Objetivos del Cliente:** La implementación del pipeline DevSecOps sobre la aplicación de microservicios resolvió con éxito integral las causas de los incidentes previos sufridos por el cliente de INSEGUS. La seguridad ha dejado de ser un obstáculo manual al final del ciclo para convertirse en una serie de **pruebas automatizadas continuas (Security as Code)**.
2. **Eficacia de los Quality Gates Diferenciados:** El uso del **Modo Aviso (Gate A)** en la compilación resolvió el conflicto clásico de bloqueo temprano en entornos vulnerables, permitiendo que la aplicación se desplegara en staging para su análisis dinámico. Simultáneamente, el **Modo Bloqueante (Gate B)** blindó la frontera productiva, certificando que ninguna versión con vulnerabilidades críticas o altas pueda alcanzar el clúster de producción.
3. **Gobierno y Trazabilidad Centralizada en DefectDojo:** La integración desatendida mediante API v2 consolidó 103 hallazgos brutos procedentes de Semgrep, Trivy y ZAP en un inventario limpio y deduplicado, permitiendo el triaje

riguroso de falsos positivos y garantizando la trazabilidad exigida por marcos de cumplimiento como ISO/IEC 27001 y el Esquema Nacional de Seguridad (ENS).

11. Referencias Normativas y Bibliográficas

(Esta sección no computa para el límite de 15 páginas según las normas del entregable)

1. **NIST Special Publication 800-181 Rev. 1:** *Workforce Framework for Cybersecurity (NICE Framework)*. National Institute of Standards and Technology, U.S. Department of Commerce.
2. **OWASP Foundation (2021):** *OWASP Top 10:2021 - The Ten Most Critical Web Application Security Risks*. Disponible en: https://owasp.org/Top10/
3. **OWASP DevSecOps Guideline:** *Automating Security Testing in Continuous Delivery Pipelines*. Open Web Application Security Project.
4. **Center for Internet Security (CIS):** *CIS Docker Benchmark v1.6.0* y *CIS Kubernetes Benchmark v1.8.0*.
5. **Semgrep Documentation:** *Writing Custom Rules for Static Analysis in JavaScript/TypeScript*. Return To Corporation, 2026.
6. **Aqua Security:** *Trivy: Comprehensive Security Scanner for Containers, File Systems and Git Repositories*. Aqua Security Software, 2026.
7. **OWASP ZAP Project:** *Zed Attack Proxy User Guide: Automated Baseline and API Scanning*. Disponible en: https://www.zaproxy.org/docs/
8. **OWASP DefectDojo Project:** *DefectDojo API v2 Documentation: Engaging, Finding and Scan Imports*. Disponible en: https://docs.defectdojo.com/
9. **ISO/IEC 27001:2022:** *Information security, cybersecurity and privacy protection — Information security management systems — Requirements*.
10. **MITRE Corporation:** *Common Weakness Enumeration (CWE) Specification: CWE-89 (SQL Injection), CWE-79 (XSS), CWE-798 (Hardcoded Credentials), CWE-918 (SSRF)*.

12. Anexos: Evidencias Técnicas

(Esta sección no computa para el límite de 15 páginas según las normas del entregable; reservada exclusivamente para evidencias técnicas)

Anexo A: Definición Completa del Pipeline GitHub Actions

Fichero: `.github/workflows/devsecops-pipeline.yml`

```
name: DevSecOps CI/CD Pipeline - INSEGUS Enterprise

on:
  push:
    branches: [ "main", "develop" ]
  pull_request:
    branches: [ "main" ]

env:
  NODE_VERSION: "18.x"
  DEFECTDOJO_URL: ${{ secrets.DEFECTDOJO_URL }}
  DEFECTDOJO_API_KEY: ${{ secrets.DEFECTDOJO_API_KEY }}

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
      - run: cd app && npm install --legacy-peer-deps && npm test

  security-static-analysis:
    needs: [build-and-test]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: SAST Semgrep
        run: docker run --rm -v $(pwd):/src semgrep/semgrep semgrep scan --config config/.semgrep.yml --json -
o evidencias/sast/semgrep_report.json app/
      - name: SCA Trivy
        run: docker run --rm -v $(pwd):/workspace aquasec/trivy:latest fs --format json -o /workspace/evidenci
as/sca/trivy_sca_report.json /workspace/app
      - name: IaC Trivy
        run: docker run --rm -v $(pwd):/workspace aquasec/trivy:latest config --format json -o /workspace/evid
encias/iac/trivy_iac_report.json /workspace/app

  quality-gate-advisory:
    needs: [security-static-analysis]
    runs-on: ubuntu-latest
    continue-on-error: true
    steps:
      - run: python3 scripts/quality_gate.py --mode advisory --semgrep evidencias/sast/semgrep_report.json --t
rivy-sca evidencias/sca/trivy_sca_report.json --trivy-iac evidencias/iac/trivy_iac_report.json

  dynamic-security-analysis:
    needs: [quality-gate-advisory]
    runs-on: ubuntu-latest
    steps:
      - name: Deploy Staging & DAST
        run: python3 scripts/run_zap_dast.py

  defectdojo-centralization:
    needs: [dynamic-security-analysis]
    runs-on: ubuntu-latest
    if: always()
    steps:
      - name: Sincronizar DefectDojo API v2
        run: |
          python3 scripts/defectdojo_client.py --action import --semgrep evidencias/sast/semgrep_report.json -
-trivy-sca evidencias/sca/trivy_sca_report.json --trivy-iac evidencias/iac/trivy_iac_report.json --zap evidenc
ias/dast/zap_report.json
          python3 scripts/defectdojo_client.py --action triage
          python3 scripts/defectdojo_client.py --action export --output evidencias/defectdojo/defectdojo_conso
lidated.json

  quality-gate-blocking:
    needs: [defectdojo-centralization]
    runs-on: ubuntu-latest
```

```
    steps:
      - name: Quality Gate Bloqueante de Release
        run: python3 scripts/quality_gate.py --mode blocking --critical 0 --high 0 --medium 10 --semgrep evide
ncias/sast/semgrep_report.json --trivy-sca evidencias/sca/trivy_sca_report.json --trivy-iac evidencias/iac/tri
vy_iac_report.json --zap evidencias/dast/zap_report.json

  deploy-to-production:
    needs: [quality-gate-blocking]
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main' && success()
    steps:
      - run: echo "Despliegue a Producción autorizado y completado."
```

Anexo B: Reglas Personalizadas SAST Semgrep

Fichero: `config/.semgrep.yml`

```
rules:
  - id: insecure-sql-string-concatenation
    pattern-either:
      - pattern: $DB.get(`... ${$VAR} ...`, ...)
      - pattern: $DB.all(`... ${$VAR} ...`, ...)
      - pattern: $DB.run(`... ${$VAR} ...`, ...)
    message: "Inyección SQL detectada (CWE-89): concatenación directa en consulta SQL."
    severity: ERROR
    languages: [javascript, typescript]
    metadata:
      cwe: "CWE-89: SQL Injection"
      owasp: "A03:2021-Injection"

  - id: hardcoded-secret-key-token
    patterns:
      - pattern-either:
          - pattern: const $SECRET = "...";
          - pattern: let $SECRET = "...";
      - metavariable-regex:
          metavariable: $SECRET
          regex: (?i)(jwt_secret|api_key|db_admin_pass|aws_secret)
    message: "Secreto o clave privada hardcodeada en el código fuente (CWE-798)."
    severity: ERROR
    languages: [javascript, typescript]

  - id: dangerous-eval-usage
    pattern: eval(...)
    message: "Uso de eval() detectado (CWE-95). Permite ejecución arbitraria de código."
    severity: ERROR
    languages: [javascript, typescript]

  - id: insecure-cors-wildcard
    pattern: "cors({ origin: '*' })"
    message: "Configuración CORS excesivamente permisiva con comodín '*' (CWE-942)."
    severity: WARNING
    languages: [javascript, typescript]
```

Anexo C: Trazas de Ejecución de Quality Gates (Aviso vs. Bloqueo Exit Code 1)

Fragmento verificado del log `evidencias/pipeline_logs/pipeline_failure_quality_gate.log`:

```
================================================================================
      INSEGUS DEVSECOPS - MOTOR DE EVALUACIÓN DE QUALITY GATES (ST3)
================================================================================
[+] MODO DE EJECUCIÓN DEL GATE: BLOCKING
--------------------------------------------------------------------------------
HERRAMIENTA / ETAPA       | CRITICAL | HIGH   | MEDIUM   | LOW
--------------------------------------------------------------------------------
SAST (Semgrep)            | 0        | 4      | 1        | 0
SCA (Trivy Dependencies)  | 3        | 47     | 32       | 7
IaC (Trivy Docker/K8s)    | 0        | 18     | 7        | 18
DAST (OWASP ZAP)          | 0        | 3      | 2        | 0
--------------------------------------------------------------------------------
TOTAL CONSOLIDADO         | 3        | 72     | 42       | 25
================================================================================

[!] MODO BLOQUEANTE (BLOCKING MODE / PRODUCTION RELEASE GATE):
    [X] ERROR CRÍTICO DE SEGURIDAD: Umbrales superados en la etapa de release:
        -> [BLOQUEADO] Vulnerabilidades CRÍTICAS (3) superan umbral (0)
        -> [BLOQUEADO] Vulnerabilidades ALTAS (72) superan umbral (0)
        -> [BLOQUEADO] Vulnerabilidades MEDIAS (42) superan umbral (10)

    [!] ACCIÓN DISCIPLINARIA: El pipeline SE DETIENE de inmediato.
    [!] El artefacto NO ha sido autorizado para despliegue en Producción.
    [!] Notificando a equipo de desarrollo y Security Team 3.

[RESULTADO] Quality Gate B (Modo Bloqueante) -> FALLIDO / BLOQUEO ACTIVO (Exit Code 1)
```

Anexo D: Exportación de Base de Datos y Triaje en DefectDojo

Fragmento del resumen consolidado extraído de `evidencias/defectdojo/defectdojo_consolidated.json`:


```
{
  "summary": {
    "total_findings": 103,
    "by_severity_total": {
      "Critical": 2,
      "High": 49,
      "Medium": 35,
      "Low": 17,
      "Info": 0
    },
    "by_severity_active": {
      "Critical": 2,
      "High": 49,
      "Medium": 34,
      "Low": 17,
      "Info": 0
    },
    "false_positives": 1,
    "mitigated": 0
  },
  "triaged_false_positives_sample": [
    {
      "id": 4,
      "title": "Semgrep: config.insecure-cors-wildcard",
      "severity": "Medium",
      "cwe": 942,
      "active": false,
      "false_p": true,
      "triage_notes": "Marcado como Falso Positivo: El endpoint expone únicamente recursos estáticos y APIs pú
blicas de prueba en red interna aislada. Se valida que el servicio no maneja cookies de sesión cruzadas (SameS
ite=Strict).",
      "triaged_by": "Security Team 3"
    }
  ]
}
```